

What is quantum computing?

A primer for people who make things — and an honest account of what these machines can and cannot do today.

You do not need to read this to use our tools. It exists for anyone who wants to know what is actually happening underneath them.

Drawn from published research by the Moth Quantum team. References at the end.

01 Why this document exists

There is a great deal of writing about quantum computing, and most of it is either too vague to be useful or too technical to be read. This is an attempt at the middle: enough detail that a physicist would not wince, plain enough that a composer or a game designer can act on it.

It is also deliberately honest about limits. Today's quantum computers are small, noisy and slow. Anyone who tells you otherwise is selling something. What makes them interesting for creative work is not that they are powerful yet, but that they are *different* — and difference is a resource that art has always been able to use before engineering could.

Three things this document tries to establish. What a quantum computer is, mechanically. Where the technology honestly stands. And why the creative industries are not a whimsical early application but arguably the correct one.

02 The machine

Bits and qubits

A classical computer stores information in bits, each definitely 0 or 1. Every image, sound file and game world you have ever encountered is a very long arrangement of those two states, transformed by operations that are, at bottom, deterministic.

A quantum computer stores information in qubits. A qubit also has two distinguishable states, written $|0\rangle$ and $|1\rangle$, but between measurements its state is described by two complex numbers — one attached to each outcome. These numbers are called amplitudes. The squares of their magnitudes give the probability of each result when you look, and so must sum to one.

That single addition — from two definite states to a pair of complex amplitudes — is the origin of everything else in this document.

Add qubits and the description grows quickly. Two qubits require four amplitudes, three require eight, and n qubits require 2^n . Fifty qubits already need more numbers than there are stars in the galaxy. This is the sense in which a quantum computer holds something a classical computer cannot: not more answers, but a larger description.

Superposition, correctly stated

A quantum computer is often described as trying every possible answer at once. This is wrong in a way that matters, and it is worth being precise because the correct version explains what quantum programming actually involves.

Before measurement, a register of qubits holds a state in which many possible outcomes have amplitudes. But the machine does not evaluate all of them and hand you the best one. If you measure such a state directly you get a single outcome, drawn at random with probability set by its amplitude — which is no more useful than guessing. The state is not a list of answers you can read. You get one sample per run, and the act of sampling destroys the state.

The work of a quantum algorithm is therefore not to search in parallel. It is to arrange, before you look, for the amplitudes of the outcomes you don't want to cancel each other out, so that the ones you do want are what remain.

The pond, and where the analogy ends

A useful picture: instead of trying answers one at a time, the machine sets up a physical situation in which every possible answer is present as a wave. The waves for wrong answers cancel each other, the way noise-cancelling headphones erase sound, while the waves for the right answer reinforce. What you measure is what survived.

Where the analogy ends: you only get to watch the pond once, and looking at it destroys it. Everything difficult about quantum programming follows from that.

Interference: the actual mechanism

Amplitudes are complex numbers, which means they carry a phase as well as a magnitude. Two amplitudes can therefore be equal and opposite, and sum to nothing.

This is the entire distinction between quantum computation and ordinary randomness. Classical probabilities only ever add — two chances of an event make it more likely, never less. Amplitudes can cancel. Every quantum algorithm that has ever been useful, from factoring to the ones in this document, works by engineering cancellation.

It is also why quantum randomness is a poor thing to build on by itself. As our Chief Science Officer puts it when asked for advice: applying a single operation, measuring it, and using the result as a random number generator misses the point entirely. It is not the randomness that is interesting. It is the structure you can impose on it.

Entanglement

Two qubits can end up in a joint state that cannot be written as one state for the first and another for the second. There is no description of the parts — only of the whole. Measure one and the possibilities available to the other change immediately, however far apart they are.

This is not a communication channel; you cannot signal with it, because the outcome you get is random. What it is, for our purposes, is a way of building correlation that has no classical equivalent: elements of a system that are genuinely inseparable rather than merely coordinated.

Measurement, and what it costs

Measurement converts a quantum state into a classical result. One outcome, probabilistically, and the superposition is gone. To learn anything statistical you must prepare the state again and meas-

ure again — many times. These repetitions are called shots, and the precision of what you learn improves only as the square root of their number. Wanting ten times the accuracy costs a hundred times the runs.

This has a direct consequence for anyone building creative tools: quantum hardware is not a function you call for a value. It is a process you sample.

What you are not allowed to do

Two prohibitions shape everything.

You cannot read a quantum state. There is no operation that reports the amplitudes; there is only measurement, which gives you a sample and destroys the rest.

And you cannot copy an unknown quantum state. This is the no-cloning theorem, and it is a genuine law rather than an engineering limitation. You can make imperfect copies — and interestingly, you can choose how the imperfection is distributed. Push the fidelity of the copy up and the original degrades correspondingly. The trade is exact, and it is the sort of constraint that turns out to be more useful to an artist than to an engineer.

Noise

Qubits are fragile. Any interaction with the surrounding environment leaks information out and destroys the phase relationships the computation depends on — a process called decoherence. Every operation is also slightly imperfect. Errors accumulate with the depth of a circuit until the output is indistinguishable from noise.

Error correction is the long-term answer, and it works by spreading one reliable logical qubit across many physical ones. It is expensive: current estimates run to hundreds or thousands of physical qubits per logical one. Nobody has a large error-corrected machine yet.

Which means that for now, noise is a property of the medium. Our own visual work leans into this deliberately — the brushes described later run with no error mitigation at all, and the imperfection of the device is part of the result.

03 Where we actually are

It helps to divide the short history of usable quantum computing into three eras. The boundaries are about what the hardware can do, and they determine what is worth building.

ERA	WHAT IT MEANS	CREATIVE CONSEQUENCE
Emulation	The quantum program is small enough that an ordinary computer can simulate it exactly.	No computational advantage. Artists found it useful anyway — a genuinely different way of transforming material.
NISQ	Noisy Intermediate-Scale Quantum. Hardware beyond what can be simulated, but not yet a machine that simply does as it is told.	Where we are. Less about advantage, still about usefulness — and where noise becomes material.
Fault tolerance	Error correction at scale. The machine does what you ask.	Where genuine speedups arrive. Hardware roadmaps point at around 2030.

The emulation ceiling

Simulating a quantum computer exactly means storing all 2^n amplitudes. At around thirty qubits that is tens of gigabytes, which is the practical limit of a good laptop; large clusters reach the mid-forties. Beyond that, exact simulation stops.

This ceiling is why emulation is not a lesser version of quantum computing but a different regime of it. Below the line, emulation is *exact* — not an approximation of the real thing but mathematically identical to a perfect device. Our own rendering research illustrates the trade precisely: the configuration in that paper needs 24 qubits and takes around twelve seconds per parameter on an Apple M3 processor. Correct, reproducible, and immediately out of reach as the problem grows.

So the honest framing of emulation is not that it is fake. It is that it is small and clean. What real hardware adds is scale and texture.

What today's machines are

Processors in the 100–160 qubit range are now routine. Our benchmarking work runs at full width on 156-qubit IBM devices, and the more interesting number it produces is not the qubit count but the depth: on one such processor we locate a critical circuit depth of around fifty, beyond which straightforward error mitigation stops recovering useful signal.

That figure is the practical constraint. Not how many qubits you can address, but how much you can do before the result dissolves. It is also why we build benchmarking tools at all — you cannot honestly claim a creative tool runs on quantum hardware without knowing what that hardware does.

Wall-clock reality

Quantum hardware is slow. Jobs are queued, shots accumulate, and a result may take minutes to hours to come back. This rules out an entire category of application: nothing per-frame, nothing per-player, nothing in the render loop.

What it permits is asynchronous work. A job during development. A job during a loading screen. A job generating the next level while the player is still in the current one. The tortoise rather than the hare — running in the background to produce the rules and structures that classical systems then execute at speed.

04 Why creative work is the right early application

The instinct that art is a soft target for a serious technology has it backwards. There are specific structural reasons the creative industries are where quantum computing becomes useful first, and they are worth stating plainly.

There are no right answers

This is the central argument. Consider cryptography: either a machine can factor numbers large enough to matter or it cannot. There is a threshold, and before it the machine is worthless for the task. The same binary applies to most proposed quantum applications — you are either past the line or you are not.

Creative work has no such threshold. There is no correct output. A generator that produces something unexpected, or textured, or strange is not failing a benchmark. It is doing the job. Which means a machine that is small, noisy and probabilistic can be genuinely useful *now*, rather than after a decade of engineering.

This is not a lowering of standards. It is a recognition that the creative industries have a different success criterion, and that this criterion happens to be compatible with the hardware that exists.

The thousand bowls of oatmeal

Procedural generation has a well-known failure mode. A system produces output that is mathematically distinct every time — different arrangements, different parameters, formally novel — and a human immediately recognises all of it as the same thing. Humans are extremely good at pattern recognition. Give them ten samples and they will describe your generator as variations on a theme.

The underlying difficulty is not a shortage of randomness. It is that useful generation means satisfying constraints while remaining varied. A level has to be completable. A melody has to resolve. A material has to be physically plausible. Ideally you would sample uniformly from the entire space of things that satisfy your constraints — but constraint satisfaction is computationally hard, so in practice people start from a working solution and perturb it. Hence the oatmeal.

Speeding up constrained sampling is one of the areas where quantum algorithms are expected to help, and it is a real problem that working developers have today.

One example, not a corpus

The dominant generative technology of the last few years requires enormous quantities of training data. That requirement has a consequence its proponents rarely dwell on: at scale, such systems must ingest the work of very large numbers of people who did not agree to it.

The methods we work with have the opposite profile. Quantum reservoir computing learns from a single example — one level, one tune — and produces variations that preserve its structure. The composer Rakhat-Bi Abdyssagin, describing a piece built this way, called it a controlled randomness: an order that evolves from chaos, where each performance is new music that is nonetheless the same composition.

That is a technical property with an ethical dividend, not the other way round. Small-data methods are what the physics affords, and it happens that they keep the artist at the centre rather than in the training set.

Noise as material

Every other industry treats hardware noise as an error budget to be suppressed. For a creative tool it can be a texture. When we ran our painting work on real hardware we deliberately omitted error mitigation, and the imperfections of the physical device became part of the aesthetic — a feature of this era of machines that we may come to miss once fault-tolerant ones arrive.

05 The mechanics, applied

Each of the phenomena in section 02 corresponds to something you can do with a tool. This is the translation layer.

PHENOMENON	CREATIVE AFFORDANCE	WHERE WE HAVE USED IT
Superposition	Holding a whole space of possible continuations rather than a shortlist, and committing only at the end.	Quantum reservoir computing for level and melody generation.
Interference	Steering a generative process by tuning which patterns survive rather than by selecting between candidates.	Coherent scattering in rendering; audio encodings.
Entanglement	Elements that cannot be described separately — colour bound to what lies beneath it, sound and image responding as one system.	The Aquarela brush, which entangles every colour along a stroke with the canvas underneath.
Measurement	The moment a generative space becomes a finished artefact.	Level generation; any point where a distribution has to become a thing.
No-cloning	A hard physical trade-off used as a control surface: fidelity of the copy against integrity of the original.	The Collage brush, which visualises the theorem directly via asymmetric cloning.
Hamiltonian evolution	Continuous, physically-grounded transformation — letting a system evolve rather than applying a filter.	The Heisenbrush, which evolves magnetisation in a spin model and produces smooth colour drift.
Amplitude damping	Decay and erasure that propagate rather than acting locally.	The Smudge brush, which attempts to remove colour and cascades it down the stroke instead.

How we decide whether the quantum is real

When designing our painting tools we set four guidelines. The effect must act locally, along the stroke. There must be a continuous control that, at zero, removes the quantum effect entirely. The algorithm must run on today's noisy hardware without error correction. And it must admit a regime in which classical simulation becomes intractable, requiring a genuine quantum processor.

The last is the honest test. A great deal of work in this field borrows the vocabulary of quantum mechanics without ever needing the machine. That criterion is how we check that we have not.

06 By medium

Visual art and images

The earliest useful creative application was image manipulation. Encode a picture into a quantum state, apply operations, decode it back — and because the operations available are not the ones in a conventional image editor, you get transformations that are difficult to obtain otherwise. This is emulation-era work, exact and fast on ordinary hardware, and it has been used in gallery contexts for several years.

Our more recent painting research goes further, treating each brush as an exploration of a specific quantum effect rather than a stylistic goal. Four brushes, four phenomena. What is notable is the order of discovery: the physics was chosen first, and the resemblances to watercolour, to Fauvist colour, to 1970s serial art emerged afterwards, unplanned.

Rendering and materials

A more technical application, and the one with the clearest path to production pipelines. Light striking a layered material can be modelled as a sequence of quantum collisions, which reproduces genuinely coherent optical behaviour — interference across many layers — that standard shading models do not reach.

The practical structure matters here: the expensive quantum part is a precomputation of optical properties, which are then baked into a conventional shader. Slow hardware is not a bottleneck if it runs once, offline, and the result is a material asset an artist uses at full speed. This is the pattern we expect most production use to follow.

Sound and music

Audio has its own encoding problem: how to represent a waveform or a score as a quantum state. There is a body of published methods for this, and our open-source audio package implements them, so that a musician or sound designer can move signals in and out of quantum representations without deriving anything.

Two directions follow. Applying quantum operations to encoded audio produces distortion and timbral effects with no conventional analogue. And reservoir methods trained on a single melody can generate endless variations that remain recognisably the same piece — which is closer to what a composer actually wants than unconstrained novelty.

Games and interactive work

Games are where the asynchronous pattern fits most naturally, because they already tolerate generation that happens between moments of play rather than during them.

We have used quantum reservoir computing to learn the structure of a hand-designed level and generate new ones — an approach applied to platform levels and to a released Roblox racing game, where courses are generated on real quantum hardware while the player is occupied with the previous one. Circuits are deliberately modest, six or seven qubits each, which keeps them well within what current devices handle reliably. A separate level-generation system has been demonstrated on hardware at scales beyond fifty qubits.

Worth noting what this is not. It is not per-frame physics and not per-player computation. The quantum part produces structures; the game engine consumes them.

07 How you actually program one

Quantum programs are conventionally written as circuits: a sequence of gates applied to specific qubits, each a precise unitary operation. This is the right abstraction for a physicist and the wrong one for almost everybody else.

The comparison we find useful: humans work at a higher level than the substrate. We compose with notes rather than instructing a piano string how to vibrate. We paint in brushstrokes rather than pixel by pixel. A game designer does not think in terms of two-qubit unitaries, and a musician asking for something louder and more distorted is not specifying a quantum channel.

Our research includes a formal answer to this. Rather than specifying gates, the programmer states where the measurable quantities should go — the expectation values — and the system works out the operations required. This turns out to be a formally universal model of quantum computation, equivalent in power to the circuit model, while never requiring the user to handle a state vector. It has been exercised on real hardware at scales beyond fifty qubits.

The point is not the formalism. It is that the interface to a quantum computer is a design problem, and treating it as one is what makes these machines available to people who are not physicists.

08 What is honestly not possible yet

A short section, and the most important one for calibrating expectations.

- **Real-time interaction.** Wall-clock latency and queueing rule out anything in a render loop or an input-response cycle.

- **Per-user computation at scale.** Individual quantum jobs for every player of a game are not realistic on current capacity.
- **General speedups.** Quantum computers are not faster at most things. Where advantage exists it is specific and structural, and for creative applications it largely remains ahead of us.
- **Large error-corrected computation.** Deep circuits still dissolve into noise. The critical depth on current hardware is around fifty layers.
- **Reading a quantum state.** Not an engineering gap but a law. You sample; you never inspect.
- **Replacing existing tools.** None of this is a substitute for a renderer, a DAW or an engine. It is an additional instrument, and it should be judged as one.

Some of our own published work sits explicitly on the near side of these limits. The rendering framework runs on an exact simulator today, with hardware execution scoped as future work. That is stated in the paper, and we would rather say it here than have someone discover it.

09 What changes with fault tolerance

Error-corrected machines are expected around the end of this decade. Two things change for creative work.

Constrained generation gets faster. The sampling problem behind the oatmeal failure — drawing varied output uniformly from the space of things satisfying a set of constraints — is a class of problem where quantum algorithms are anticipated to help substantially. If that holds, the effect on procedural generation across games, film and interactive media is not incremental.

New mathematics becomes available as a generator. Some quantities can be computed exponentially faster on a fault-tolerant machine than classically. Topological invariants are one example, and our vision work ends with a designed-but-unbuilt game whose central mechanic depends on evaluating one of them — a game that is, by construction, impossible without a quantum computer. That is what an application looks like when it genuinely requires the hardware rather than decorating itself with it.

The strategy this implies is two-pronged and we state it openly: build everything that is possible on the machines that exist, and develop the algorithms for the machines that are coming.

Where this era sits, historically

In 1962 a group at MIT wrote *Spacewar!* on a machine that cost more than a house. It did not justify the expense. What it proved was that a computer could produce an experience unobtainable any other way. *Pong* arrived ten years later.

Quantum computing is somewhere around 1962. The work of this period is not to demonstrate economic advantage; it is to establish that something new is possible. We would rather be doing that than waiting.

10 Glossary

Amplitude

A complex number attached to a possible measurement outcome. Its magnitude squared gives the probability of that outcome.

Circuit depth

The number of sequential operation layers in a quantum program. The practical limit on current hardware, more binding than qubit count.

Decoherence

Loss of quantum behaviour through interaction with the environment. The reason quantum computers need to be isolated and cold.

Emulation

Simulating a quantum computer on a classical one. Exact below roughly thirty qubits; impossible above the mid-forties.

Entanglement

A joint state of two or more qubits that cannot be decomposed into separate states for each.

Error mitigation

Statistical post-processing that partially compensates for hardware noise. Distinct from error correction, which prevents it.

Expectation value

The average of a measurement over many repetitions. What you actually get from hardware, rather than a state.

Interference

Amplitudes adding and cancelling. The mechanism by which quantum computation produces useful answers.

NISQ

Noisy Intermediate-Scale Quantum. The current era: too large to simulate, too noisy to be reliable.

No-cloning theorem

The impossibility of copying an unknown quantum state exactly. Imperfect copies are possible, with an exact trade-off against the original.

Qubit

The basic unit of quantum information. Two distinguishable states, and a continuum of superpositions between them.

Quantum reservoir computing

A machine-learning method that uses a quantum system as a fixed nonlinear transformation, with only a small classical output layer trained. Learns from very little data.

Shot

One preparation-and-measurement cycle. Statistics require many; precision improves only as the square root of their number.

Superposition

A state holding several possible outcomes with amplitudes. Not a set of answers that can be read.

11 Further reading

MOTH QUANTUM RESEARCH

1. **Rendering Coherent Scattering via Quantum Collision Models.** Ferreira, Topel, Fromholz, Wootton. arXiv:2606.29989. Light-matter interaction as quantum collisions; materials with optical behaviour classical shading cannot reach.
2. **QPU-scale randomized benchmarking via Bell-pair injection.** Pettugani, Aguado-Yáñez, Park, Bultrini, Wootton. arXiv:2606.20123. Measuring what a whole processor can do, and locating where it stops working.
3. **Quantum circuit design via dynamic Pauli constraints.** Wootton, Incerti-Medici, Bultrini, Fromholz. arXiv:2605.22744. A universal model of quantum computation designed around usability rather than gates.
4. **Quantum Brush: a quantum computing-based tool for digital painting.** Ferreira, Crippa, Park, Bultrini, Fromholz, Lipski, Jansen, Wootton. arXiv:2509.01442. Four brushes, four quantum effects, run on hardware with the errors left in.
5. **Procedural Generation and Games at the Dawn of Fault Tolerant Quantum Computing.** Bultrini, Wootton. arXiv:2508.09683. Which parts of procedural generation have a real speedup waiting for them.
6. **Level Generation with Quantum Reservoir Computing.** Ferreira, Fromholz, Shaji, Wootton. arXiv:2505.13287. Learning the structure of one level and generating more.

BACKGROUND, WRITTEN FOR NON-SPECIALISTS

1. **Painting with Quantum Physics.** Moth Quantum, September 2025. The four brushes explained without equations.
2. **A new way to think about quantum programming.** Moth Quantum, May 2026. Why the circuit model is the wrong interface for creative work.
3. **Benchmarking a decade of quantum computers with games.** Moth Quantum, August 2025. How the benchmarking method works, and why it is playable.
4. **Composing for Quantum Games.** Moth Quantum. Generating a soundtrack from quantum output, with the composer's own account.

This document summarises published work by the Moth Quantum research team. Where figures are cited — circuit depths, qubit counts, execution times — they are drawn from the papers listed above and are accurate as of publication. Hardware capability in this field changes quickly; the papers, not this primer, are the record.